SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	R SARVESH	**Reg. No.:**	192521143
Section / Batch:	B.TECH IT	**Date:**	27/7/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I R SARVESH (192521143) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: R SARVESH

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

CSA0620-DESIGN AND ANALYSIS OF ALGORITHM

NAME: R SARVESH

REG. NO: 192521143

DEPT.: B.TECH INFORMATION TECHNOLOGY

CO2 ASSESSMENT TOOL 2

---

1. A courier service calculates distances between pickup and delivery points using brute-force to assign the nearest courier. a) Explain how the brute-force closest pair algorithm works. b) Derive the time complexity. c) Discuss scalability issues as the number of requests increases.

a. The brute-force closest pair algorithm finds the minimum distance between pickup and delivery points by checking every possible pair of points.

Steps:

1. Store all courier locations as coordinate points (x, y).
2. Calculate the distance between every pair of points using the Euclidean distance formula.
3. Compare all distances.
4. Select the pair having the smallest distance.

Distance formula:

$$d = \left((x2 - x1)^2 + (y2 - y1)\right)^{\frac{1}{2}}$$

b. Time Complexity Derivation

For n courier locations:

- Number of possible pairs: n(n-1)/2
- Distance calculation takes constant time O(1).

Therefore:

$T(n)=n(n-1)/2\times O(1)$
$T(n)= O(n^2)$

c. Scalability Issues

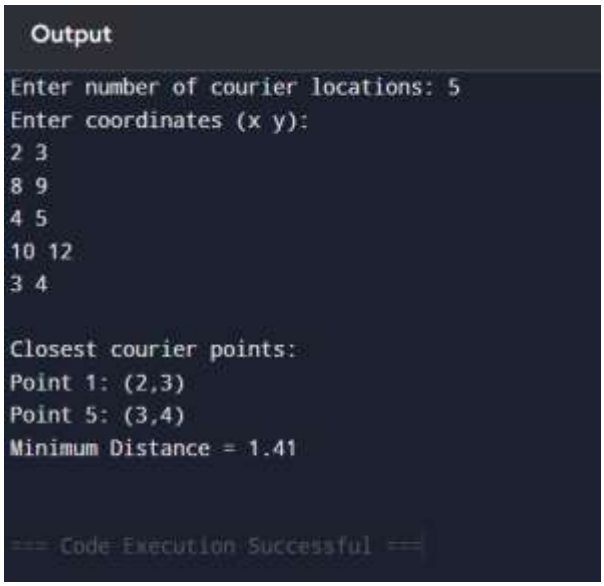
- The number of distance calculations increases rapidly as requests grow.
- For large courier networks, brute-force becomes slow.
- Real-time courier assignment becomes inefficient.
- Advanced methods like divide-and-conquer closest pair algorithms can improve performance.

C Program: Brute-Force Closest Pair

```
#include <stdio.h>
#include <math.h>

struct Point {
   int x;
   int y;
};

double distance(struct Point p1, struct Point p2)
{
   return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2));
}

int main()
{
   int n;

   printf("Enter number of courier locations: ");
   scanf("%d", &n);

   struct Point points[n];

   printf("Enter coordinates (x y):\n");

   for(int i = 0; i < n; i++)
   {
      scanf("%d %d", &points[i].x, &points[i].y);
```

```
    }

    double minDistance = INFINITY;
    int point1 = 0, point2 = 0;

    for(int i = 0; i < n-1; i++)
    {
        for(int j = i+1; j < n; j++)
        {
            double d = distance(points[i], points[j]);

            if(d < minDistance)
            {
                minDistance = d;
                point1 = i;
                point2 = j;
            }
        }
    }

    printf("\nClosest courier points:\n");
    printf("Point %d: (%d,%d)\n", point1+1, points[point1].x, points[point1].y);
    printf("Point %d: (%d,%d)\n", point2+1, points[point2].x, points[point2].y);

    printf("Minimum Distance = %.2f\n", minDistance);

    return 0;
}
```

Output:



2. A disaster response system maps affected zones using coordinate points and determines the boundary using convex hull techniques. a) Explain how brute-force convex hull operates. b) Analyze its computational complexity. c) Discuss limitations when handling large datasets.

a. The brute-force convex hull algorithm finds the boundary points enclosing all disaster-affected zones.

Steps:

1. Consider every pair of points as a possible boundary edge.
2. Check whether all remaining points lie on the same side of the selected edge.
3. If all points satisfy the condition, the edge belongs to the convex hull.
4. Collect all such boundary points.

b. For n coordinate points:

- Possible pairs of points: O(n^2)
- Checking all remaining points for each pair: O(n)
- Total complexity:

  O(n^2) x O(n) = O(n^3)

c) Limitations for Large Datasets

- Requires checking every possible point combination.
- Execution time increases rapidly with more affected zones.
- Not suitable for real-time disaster mapping.
- Memory and computation requirements become high.
- Efficient algorithms like Graham Scan and QuickHull are preferred.

C Program: Brute-Force Convex Hull

```
#include <stdio.h>

struct Point {
   int x;
   int y;
};

int orientation(struct Point a, struct Point b, struct Point c)
{
   int value = (b.y - a.y) * (c.x - b.x) -
```

```
            (b.x - a.x) * (c.y - b.y);

    if(value == 0)
        return 0;

    return (value > 0) ? 1 : 2;
}

int main()
{
    int n;

    printf("Enter number of points: ");
    scanf("%d", &n);

    struct Point points[n];

    printf("Enter coordinates (x y):\n");

    for(int i = 0; i < n; i++)
    {
        scanf("%d %d", &points[i].x, &points[i].y);
    }

    printf("\nConvex Hull Boundary Points:\n");

    for(int i = 0; i < n; i++)
    {
        for(int j = i+1; j < n; j++)
        {
            int left = 0, right = 0;

            for(int k = 0; k < n; k++)
            {
                if(k == i || k == j)
                    continue;

                int o = orientation(points[i], points[j], points[k]);

                if(o == 1)
                    left++;

                else if(o == 2)
                    right++;
            }
```

```
            if(left == 0 || right == 0)
            {
                printf("(%d,%d) - (%d,%d)\n",
                points[i].x, points[i].y,
                points[j].x, points[j].y);
            }
        }
    }

    return 0;
}
```

Output: